

Sprint 10: Data Preprocessing

1. Data Cleaning Techniques and Handling Missing Values

○ Introduction to Data Cleaning:

- The importance of data cleaning in the data preprocessing pipeline.
- Common data quality issues: duplicates, incorrect data types, outliers, and inconsistencies.

○ Handling Missing Values:

- Understanding types of missing data: MCAR (Missing Completely at Random), MAR (Missing at Random), MNAR (Missing Not at Random).
- Techniques for handling missing values:
 - **Removing Missing Data:** When and how to drop rows or columns with missing values.
 - **Imputation Methods:**
 - Simple imputation: Mean, median, and mode.
 - Advanced imputation: KNN imputation, multivariate imputation, and regression imputation.
 - **Creating Indicator Variables:** Use of binary flags to indicate the presence of missing data.

2. Data Scaling and Normalization

○ Importance of Data Scaling:

- How scaling affects the performance of machine learning algorithms, especially those based on distance or gradient descent.

○ Scaling Techniques:

- **Min-Max Scaling:** Rescaling data to a fixed range, typically 0 to 1.
- **Z-Score Normalization (Standardization):** Adjusting data to have a mean of 0 and a standard deviation of 1.
- **Robust Scaling:** Scaling using the median and interquartile range,

effective for data with outliers.

- **When to Use Each Technique:** Practical guidelines for choosing the appropriate scaling method.

3. Introduction to Feature Engineering

- **Definition and Importance:**
 - Transforming raw data into features that better represent the underlying problem to the predictive models.
- **Techniques in Feature Engineering:**
 - **Creating New Features:** Deriving new features from existing ones (e.g., combining or transforming variables).
 - **Encoding Categorical Variables:** Techniques like one-hot encoding, label encoding, and target encoding.
 - **Interaction Features:** Creating features by combining multiple variables to capture their interaction.
 - **Polynomial Features:** Generating polynomial combinations of features to model non-linear relationships.

Few words about scikit-learn

We will use **scikit-learn**, often abbreviated as **sklearn**, - one of the most popular and powerful open-source libraries in Python for machine learning and data analysis. It provides a comprehensive suite of tools for building, evaluating, and deploying machine learning models, making it an essential tool for data scientists, machine learning engineers, and statisticians.

Key Features of **scikit-learn**:

1. Machine Learning Algorithms:

- **scikit-learn** includes a wide array of machine learning algorithms for both supervised and unsupervised learning tasks. This includes algorithms for:
 - **Classification:** Logistic Regression, Support Vector Machines (SVM),

Random Forests, k-Nearest Neighbors (k-NN), Naive Bayes, etc.

- **Regression:** Linear Regression, Ridge Regression, Lasso, Decision Trees, etc.
- **Clustering:** K-Means, Hierarchical Clustering, DBSCAN, etc.
- **Dimensionality Reduction:** Principal Component Analysis (PCA), t-SNE, etc.
- **Model Selection and Evaluation:** Tools like Grid Search, Cross-Validation, and metrics such as accuracy, precision, recall, F1-score, and more.

2. Preprocessing Tools:

- `scikit-learn` offers a variety of preprocessing utilities for preparing data before it is fed into a machine learning model. This includes:
 - **Data Scaling and Normalization:** `StandardScaler`, `MinMaxScaler`, `RobustScaler`.
 - **Encoding Categorical Variables:** `OneHotEncoder`, `LabelEncoder`.
 - **Feature Engineering:** Polynomial features, interaction terms, and more.
 - **Imputation:** `SimpleImputer`, `KNNImputer` for handling missing data.

3. Model Validation and Selection:

- `scikit-learn` provides powerful tools for model validation and hyperparameter tuning, ensuring that your models generalize well to unseen data.
 - **Cross-Validation:** Techniques like k-fold cross-validation, stratified k-fold, and more.
 - **Grid Search and Random Search:** `GridSearchCV`, `RandomizedSearchCV` to find the best hyperparameters for your models.

4. Pipelines:

- Pipelines in `scikit-learn` allow you to streamline the process of

creating machine learning workflows by combining multiple steps (e.g., preprocessing, model fitting) into a single, cohesive process.

■ **Example:**

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression())
])

pipeline.fit(X_train, y_train)
```

5. Model Evaluation Metrics:

- `scikit-learn` includes a rich set of metrics for evaluating model performance, from standard accuracy to more complex metrics like ROC AUC, mean squared error (MSE), and adjusted R^2 .

■ **Example:**

```
from sklearn.metrics import accuracy_score, confusion_matrix

y_pred = pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
```

6. Ease of Use:

- `scikit-learn` is designed with simplicity in mind, using consistent and intuitive APIs. It integrates seamlessly with other popular Python libraries such as NumPy, SciPy, and Pandas, allowing for easy data manipulation and visualization.

7. Extensive Documentation and Community Support:

- The library is well-documented, with extensive tutorials, examples, and user guides available. It also has a large and active community, making it easy to find help and resources.

Use Cases:

- **Data Preprocessing:** Preparing raw data for analysis, including handling missing values, scaling, and encoding.
- **Building Predictive Models:** Developing models for tasks such as predicting customer churn, classifying images, or forecasting sales.
- **Feature Engineering:** Enhancing the predictive power of datasets by creating new features or transforming existing ones.
- **Model Evaluation:** Assessing the performance of machine learning models to ensure they generalize well to new data.
- **Hyperparameter Tuning:** Finding the best parameters for machine learning models to optimize their performance.

✓ 1. Data Cleaning Techniques and Handling Missing Values

In the real world, raw data is rarely perfect. It often contains errors, inconsistencies, and missing values. Data cleaning is the process of detecting and correcting these issues to improve the quality of the data, which is crucial for building reliable machine learning models.

1.1. Common Data Issues

- **Duplicates:** These are repeated entries in the dataset. Duplicate rows can skew your analysis or model, especially if they represent the same event or entity.
- **Incorrect Data Types:** Data that is incorrectly typed (e.g., numbers stored as strings) can lead to errors in analysis and model training.
- **Outliers:** Extreme values that are far removed from the rest of the data. They can distort the results of statistical analyses or machine learning models.

1.2. Handling Missing Values

Missing data can occur for various reasons, such as data entry errors, equipment malfunctions, or privacy concerns. It's essential to handle missing values carefully, as improper handling can lead to biased results.

Types of Missing Data:

- **MCAR (Missing Completely at Random):** The probability of data being missing is independent of the data itself.
- **MAR (Missing at Random):** The probability of missing data is related to other observed data, but not to the missing data itself.
- **MNAR (Missing Not at Random):** The missingness is related to the unobserved data itself.

✓ 1.3. Techniques for Handling Missing Values

- **Removing Missing Data:** Dropping rows or columns with missing values is the simplest approach, but it can lead to loss of valuable information.

```
#You need this code block only in case using Google Colaboratory
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/My Drive/Colab Notebooks/MLandDS/
```

```
Mounted at /content/drive
/content/drive/My Drive/Colab Notebooks/MLandDS
```

```
import pandas as pd

df = pd.read_csv('data.csv')
df_cleaned = df.dropna() # Drops all rows with any missing values
# Drop rows with any missing values in numerical columns
df = df.dropna(subset=['feature1', 'feature2'])
```

Imputation Methods:

- **Mean Imputation:** Replace missing values with the mean of the column.

```
# Impute missing values with the mean of the respective column
df['feature1'].fillna(df['feature1'].mean(), inplace=True)
df['feature2'].fillna(df['feature2'].mean(), inplace=True)
```

- **Median Imputation:** Useful when the data is skewed.

```
df['feature1'].fillna(df['feature1'].median(), inplace=True)
```

- **Mode Imputation:** Often used for categorical data.

```
df['feature1'].fillna(df['feature1'].mode()[0], inplace=True)
```

- **Advanced Imputation:**

- **K-Nearest Neighbors (KNN) Imputation:** Estimates missing values based on the values of the nearest neighbors.
- **Multivariate Imputation:** Considers the relationships between features to impute missing values.

Creating Indicator Variables: Another approach is to create a binary indicator variable that flags the presence of missing data.

```
df['missing_column'] = df['feature1'].isnull().astype(int)
```

✓ 2. Data Scaling and Normalization

In machine learning, data scaling and normalization are crucial preprocessing steps that can significantly affect the performance of your models. These techniques are particularly important for algorithms that rely on distance metrics (e.g., K-Nearest Neighbors, Support Vector Machines) or gradient-based optimization methods (e.g., linear regression, neural networks). Below, we will explore why scaling is important, different scaling techniques, and when to use each technique.

2.1. Importance of Data Scaling

Data scaling ensures that all features contribute equally to the model's learning process. When features have vastly different ranges, those with larger ranges can dominate the distance calculations or the gradient steps, leading to suboptimal models. For instance:

- **Distance-based Algorithms:** In algorithms like K-Nearest Neighbors (KNN) or Support Vector Machines (SVM), the distance between data points is crucial. If one feature has a much larger range than others, it will disproportionately influence the distance metric, potentially skewing the results.
- **Gradient Descent:** Algorithms like linear regression and neural networks use gradient descent to minimize a cost function. If the features are not scaled, the gradient descent algorithm may take longer to converge or may converge to a suboptimal solution due to the uneven steps taken in the parameter space.

2.2. Scaling Techniques

Min-Max Scaling

Min-Max Scaling (or normalization) rescales the data to a fixed range, usually [0, 1]. This technique is particularly useful when you know that your data follows a distribution with clear upper and lower bounds.

Formula:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Where:

- X is the original value.
- $X_{\min}$ and $X_{\max}$ are the minimum and maximum values of the feature.

Example:

Suppose you have a feature with values ranging from 10 to 200. Applying Min-Max Scaling to this feature will rescale it to a range of [0, 1].

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd
```

```
data = {'feature': [10, 50, 100, 150, 200]}
df = pd.DataFrame(data)
```

```
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)
```

```
print(scaled_data)
```

```
[[0.         ]
 [0.21052632]
 [0.47368421]
 [0.73684211]
 [1.         ]]
```

This rescaled feature now lies between 0 and 1.

One more example of **Min-Max Scaling**:

```
# Select numerical columns for scaling
numerical_cols = df[['feature1', 'feature2']]

# Keep categorical and target columns separate
categorical_cols = df[['categorical_column', 'target']]
```

```
from sklearn.preprocessing import MinMaxScaler

# Initialize the scaler
scaler = MinMaxScaler()

# Fit and transform the numerical columns
numerical_scaled = scaler.fit_transform(numerical_cols)

# Convert the scaled data back to a DataFrame
numerical_scaled_df = pd.DataFrame(numerical_scaled, columns=numerica
```

```
# Combine scaled numerical data with the categorical columns
df_scaled = pd.concat([numerical_scaled_df, categorical_cols.reset_in

# Display the final DataFrame
print(df_scaled)
```

	feature1	feature2	categorical_column	target
0	0.000000	0.567568	A	10
1	0.268293	0.584459	B	15
2	0.463415	0.000000	C	7
3	0.853659	1.000000	A	18
4	0.571816	0.432432	B	11
5	0.951220	0.837838	C	19
6	0.390244	0.584459	A	13
7	0.707317	0.756757	B	16
8	1.000000	0.810811	NaN	14
9	0.512195	0.270270	C	12

Use Case:

- Use Min-Max Scaling when you want all your features to have the same scale (e.g., in algorithms like KNN, SVM).
- Particularly useful when the data is distributed across a known and fixed range.

✓ Z-Score Normalization (Standardization)

Z-Score Normalization (or standardization) rescales the data to have a mean of 0 and a standard deviation of 1. This technique is useful when the data follows a Gaussian (normal) distribution but not necessarily bounded.

Formula:

$$X' = \frac{X - \mu}{\sigma}$$

Where:

- μ is the mean of the feature.
- σ is the standard deviation of the feature.

Example:

Suppose you have a feature with values [10, 50, 100, 150, 200]. Applying Z-Score Normalization will adjust the data to have a mean of 0 and a standard deviation of 1.

```
from sklearn.preprocessing import StandardScaler
import pandas as pd
```

```
data = {'feature': [10, 50, 100, 150, 200]}
df = pd.DataFrame(data)
```

```
scaler = StandardScaler()
standardized_data = scaler.fit_transform(df)
```

```
print(standardized_data)
```

```
[[-1.35411306]
 [-0.76536825]
 [-0.02943724]
 [ 0.70649377]
 [ 1.44242478]]
```

One more example of **Z-Score Normalization (Standardization)**:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_standardized = scaler.fit_transform(numerical_cols)
```

This rescaled feature now has a mean of 0 and a standard deviation of 1.

Use Case:

- Use Z-Score Normalization when your data follows a normal distribution.
- It's the default choice for many machine learning models, especially those assuming normally distributed data.

✓ **Robust Scaling**

Robust Scaling uses the median and interquartile range (IQR) for scaling, which makes it robust to outliers. Unlike Min-Max Scaling and Z-Score Normalization, Robust Scaling is less influenced by extreme values in the data.

Formula:

$$X' = \frac{X - \text{median}(X)}{\text{IQR}(X)}$$

Where:

- $\text{IQR}(X)$ is the range between the 25th percentile ($Q1$) and the 75th percentile ($Q3$).

Example:

Suppose you have a feature with values [10, 50, 100, 150, 200]. Robust Scaling will adjust the data based on the median and **IQR**.

```
from sklearn.preprocessing import RobustScaler
import pandas as pd

data = {'feature': [10, 50, 100, 150, 200]}
df = pd.DataFrame(data)

scaler = RobustScaler()
robust_scaled_data = scaler.fit_transform(df)

print(robust_scaled_data)
```

```
[[-0.9]
 [-0.5]
 [ 0. ]
 [ 0.5]
 [ 1. ]]
```

This scaling ensures that outliers have less influence on the scaled data.

One more example of **Robust Scaling**:

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
df_robust_scaled = scaler.fit_transform(numerical_cols)
```

Use Case:

- Use Robust Scaling when your data contains outliers or follows a non-Gaussian distribution.
- Ideal for situations where the data is skewed or contains extreme values that should not heavily influence the scaling.

✓ 2.3. When to Use Each Technique

- **Min-Max Scaling:**

- Use when you want to bring all features into the same scale, especially for algorithms that rely on distance metrics (e.g., KNN, SVM).
- Suitable for data with known bounds.

- **Z-Score Normalization (Standardization):**

- Use when your data follows a normal distribution, or when using models that assume normally distributed data (e.g., linear regression, logistic regression).
- It's also a good general-purpose scaler when you don't have specific needs for Min-Max Scaling or Robust Scaling.

- **Robust Scaling:**

- Use when your data contains outliers or is heavily skewed.
- Ideal for datasets with features that have varying scales and are not normally distributed.

Scaling and normalization are critical steps in the data preprocessing pipeline. The choice of technique depends on the nature of your data and the requirements of the machine learning algorithm you are using. By appropriately scaling your data, you ensure that your models perform optimally and converge more quickly during training.

c. Choosing the Right Scaling Technique:

- **Min-Max Scaling:** Use when you know the boundaries of your data or when the algorithm requires data in a specific range.
- **Z-Score Normalization:** Suitable for normally distributed data or when you want to standardize features to have equal importance.
- **Robust Scaling:** Best when dealing with data that has outliers.

✓ 3. Introduction to Feature Engineering

Feature engineering is a crucial step in the data preprocessing pipeline. It involves transforming raw data into features that better represent the underlying problem to the predictive models. Good feature engineering can significantly improve the performance of machine learning models by providing them with the most relevant and informative input data.

3.1. Definition and Importance

Feature Engineering refers to the process of selecting, modifying, or creating new features from raw data to improve the performance of machine learning models. The goal is to create features that capture the underlying patterns in the data more effectively than the original raw data. Proper feature engineering can lead to simpler models, improved accuracy, and reduced training times.

3.2. Techniques in Feature Engineering

Creating New Features

Creating new features involves deriving new variables from existing ones that better represent the underlying structure of the data. This could involve mathematical transformations, aggregations, or domain-specific operations.

Example 1: Mathematical Transformations

Suppose you have a dataset with the `height` and `weight` of individuals. You can create a new feature, Body Mass Index (BMI), which is a better indicator of health than either `height` or `weight` alone.

```
import pandas as pd
```

```
# Example data
data = {
    'height': [1.60, 1.75, 1.82, 1.90],
    'weight': [55, 80, 72, 90]
}
df = pd.DataFrame(data)

# Creating a new feature: BMI
df['BMI'] = df['weight'] / (df['height'] ** 2)

print(df)
```

	height	weight	BMI
0	1.60	55	21.484375
1	1.75	80	26.122449
2	1.82	72	21.736505
3	1.90	90	24.930748

Example 2: Date-Time Features

Suppose you have a `date` feature. You can create new features such as `day_of_week`, `month`, or `year` to capture patterns that occur over time.

```
# Example data
data = {'date': ['2024-01-01', '2024-06-15', '2024-12-31']}
df = pd.DataFrame(data)

# Convert to datetime
df['date'] = pd.to_datetime(df['date'])

# Creating new date-time features
df['day_of_week'] = df['date'].dt.day_name()
df['month'] = df['date'].dt.month
df['year'] = df['date'].dt.year

print(df)
```

	date	day_of_week	month	year
0	2024-01-01	Monday	1	2024
1	2024-06-15	Saturday	6	2024
2	2024-12-31	Tuesday	12	2024

✓ Encoding Categorical Variables

Machine learning algorithms generally require numerical input. Therefore, categorical variables must be converted into numerical format. There are several techniques to encode categorical variables:

a. One-Hot Encoding

One-hot encoding converts each category into a new binary column. This method is suitable when there are a limited number of categories.

Example:

```
# Example data
data = {'color': ['red', 'blue', 'green']}
df = pd.DataFrame(data)

# One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=['color'])

print(df_encoded)
```

	color_blue	color_green	color_red
0	False	False	True
1	True	False	False
2	False	True	False

b. Label Encoding

Label encoding assigns a unique integer to each category. It is simple but assumes an ordinal relationship between the categories, which may not be appropriate for all cases.

Example:

```
from sklearn.preprocessing import LabelEncoder

# Example data
data = {'color': ['red', 'blue', 'green']}
df = pd.DataFrame(data)

# Label Encoding
encoder = LabelEncoder()
df['color_encoded'] = encoder.fit_transform(df['color'])

print(df)
```

	color	color_encoded
0	red	2
1	blue	0
2	green	1

c. Target Encoding

Target encoding involves replacing each category with the mean of the target variable for that category. This technique is useful when the categorical variable has a large number of levels.

Example:

```
# Example data
data = {
    'color': ['red', 'blue', 'green', 'red', 'blue', 'green'],
    'target': [1, 0, 0, 1, 1, 0]
}
df = pd.DataFrame(data)

# Target Encoding
df['color_encoded'] = df.groupby('color')['target'].transform('mean')

print(df)
```

	color	target	color_encoded
0	red	1	1.0
1	blue	0	0.5
2	green	0	0.0
3	red	1	1.0
4	blue	1	0.5
5	green	0	0.0

Interaction Features

Interaction features are created by combining two or more variables to capture the interactions between them. These features can reveal relationships that are not apparent when considering the variables independently.

Example:

Suppose you have `age` and `income` as features. An interaction term could be created by multiplying these two features to capture how income changes with age.

```
# Example data
data = {
    'age': [25, 35, 45, 55],
    'income': [30000, 50000, 70000, 90000]
}
df = pd.DataFrame(data)

# Creating an interaction feature
df['age_income_interaction'] = df['age'] * df['income']

print(df)
```

	age	income	age_income_interaction
0	25	30000	750000
1	35	50000	1750000
2	45	70000	3150000
3	55	90000	4950000

Polynomial features are created by generating new features that are polynomial combinations of the existing features. This can be particularly useful when modeling non-linear relationships.

Example:

Suppose you have a single feature x . You can create polynomial features like x^2 , x^3 , etc., which may help in capturing the non-linear relationships in the data.

```
from sklearn.preprocessing import PolynomialFeatures

# Example data
data = {'x': [2, 3, 4]}
df = pd.DataFrame(data)

# Creating polynomial features (degree 2)
poly = PolynomialFeatures(degree=2, include_bias=False)
poly_features = poly.fit_transform(df)

# Convert the polynomial features back to a DataFrame
df_poly = pd.DataFrame(poly_features, columns=poly.get_feature_names_out())

print(df_poly)
```

	x	x^2
0	2.0	4.0
1	3.0	9.0
2	4.0	16.0

In this example, the original feature x is transformed into two features: x and x^2 , allowing a model to capture quadratic relationships in the data.

✓ Target Encoding with Smoothing

Target encoding transforms categorical variables by replacing each category with the mean of the target variable for that category. While this method can capture the relationship between categorical features and the target, it may lead to overfitting, especially when some categories have few observations.

Smoothing addresses this issue by balancing the category-specific mean with the overall mean of the target variable. This approach ensures that categories with limited data are not assigned misleadingly extreme values.

Smoothing Formula

The smoothed target encoding for a category is calculated as:

$$\text{Encoding} = \frac{n \cdot \text{Mean}_{\text{category}} + m \cdot \text{Mean}_{\text{global}}}{n + m}$$

Where:

- n is the number of observations for the category.
- $\text{Mean}_{\text{category}}$ is the mean target value for the category.
- $\text{Mean}_{\text{global}}$ is the overall mean target value across all categories.
- m is the smoothing parameter that determines the weight of the global mean.

A higher m value increases the influence of the global mean, which is particularly useful for categories with few observations.

Implementation in Python

Here's how you can implement smoothed target encoding using `pandas`

```
import pandas as pd

def target_encode_smooth(df, cat_col, target_col, m):
    # Compute the global mean
    global_mean = df[target_col].mean()

    # Compute the number of observations and the mean for each category
    agg = df.groupby(cat_col)[target_col].agg(['count', 'mean'])

    # Compute the smoothed means
    smooth = (agg['count'] * agg['mean'] + m * global_mean) / (agg['count'] + m)

    # Map the smoothed means to the original dataframe
    return df[cat_col].map(smooth)
```

This function calculates the smoothed target encoding for a specified categorical column. You can adjust the smoothing parameter m based on your dataset's characteristics.

More examples with **Feature engineering**.

a. Creating New Features:

- **Polynomial Features:** Creating polynomial combinations of existing features to capture non-linear relationships.

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2)
df_poly = poly.fit_transform(numerical_cols)
```

- **Interaction Features:** Combining two or more features to capture interactions.

```
df['new_feature'] = df['feature1'] * df['feature2']
```

b. Encoding Categorical Variables:

- **One-Hot Encoding:** Converts categorical variables into a series of binary columns.

```
df_encoded = pd.get_dummies(df, columns=['categorical_column'])
```

- **Target Encoding:** Replaces categorical values with the mean of the target variable for that category.

```
df['encoded_column'] = df.groupby('categorical_column')['target'].transform('mean')
```

c. Feature Selection:

- **Correlation Analysis:** Identify features that are highly correlated with the target variable.

```
# Select only numerical columns for correlation
numerical_df = df.select_dtypes(include=[float, int])

# Calculate the correlation matrix
correlation_matrix = numerical_df.corr()

# Print the correlations with the 'target' variable, sorted in descending order
print(correlation_matrix['target'].sort_values(ascending=False))
```

```
target          1.000000
feature2        0.870459
new_feature      0.827238
feature1         0.606236
encoded_column   0.153741
missing_column   NaN
Name: target, dtype: float64
```


Feature engineering is a powerful tool in the data preprocessing process. By creating new features, encoding categorical variables, generating interaction terms, and adding polynomial features, you can significantly enhance the performance of machine learning models. The effectiveness of feature engineering often depends on the domain knowledge and creativity of the practitioner, as it requires understanding the underlying data and the problem being solved.

Start coding or generate with AI.